

E0 261 Sample Questions (MidTerm)

1. Consider the following SQL query:

*select * from EMP where EMP.salary > 40000 AND EMP.age = 30*

Assume that there is a clustered B+ tree index (I_1) on *EMP.salary* and an unclustered B+ tree index (I_2) on *EMP.age*. The employee salaries range from 20001 till 100000 and the ages range from 21 till 60. The statistics are as follows:

NCARD = 100000 tuples, TCARD = 1000 pages, $P = 0.8$,

ICARD(I_1) = 1000 values, NINDX(I_1) = 500 pages, ICARD(I_2) = 40 values, NINDX(I_2) = 200 pages.

Assume that we are concerned only with the IO cost and that the buffer pool allocated to this query can hold 1200 pages.

1. Calculate the tuple cardinality estimated by System R for the query's result. (2 marks)
2. What access paths would be considered by the optimizer for the relation *EMP*? Which path is chosen? Show the cost calculations. (4 marks)

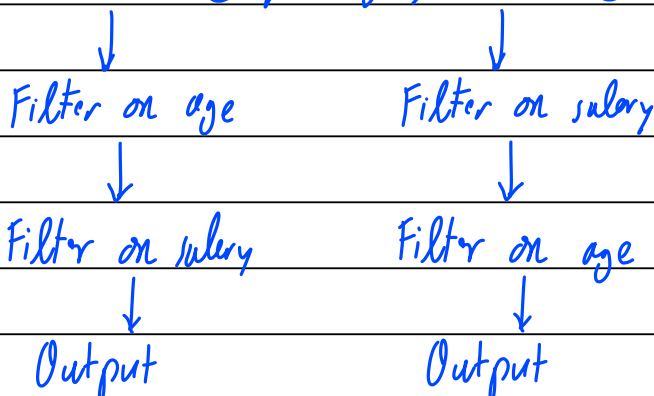
1) Assuming uniform distribution,

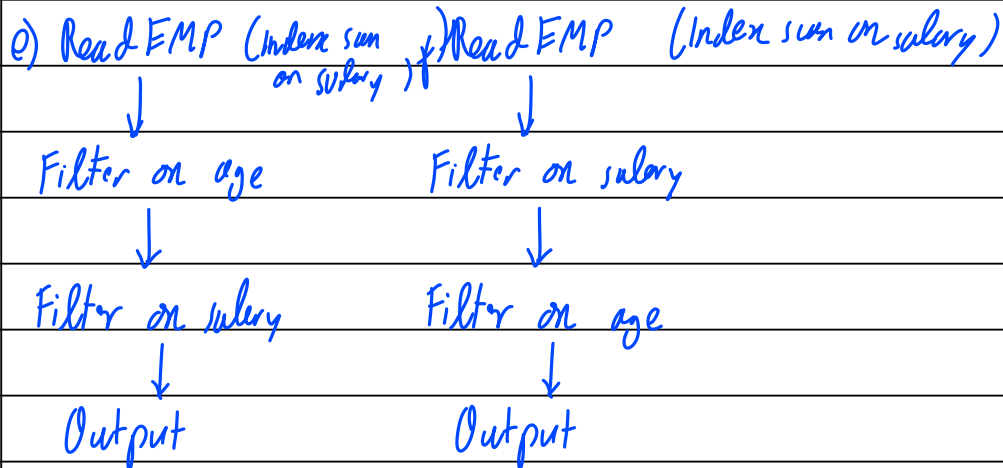
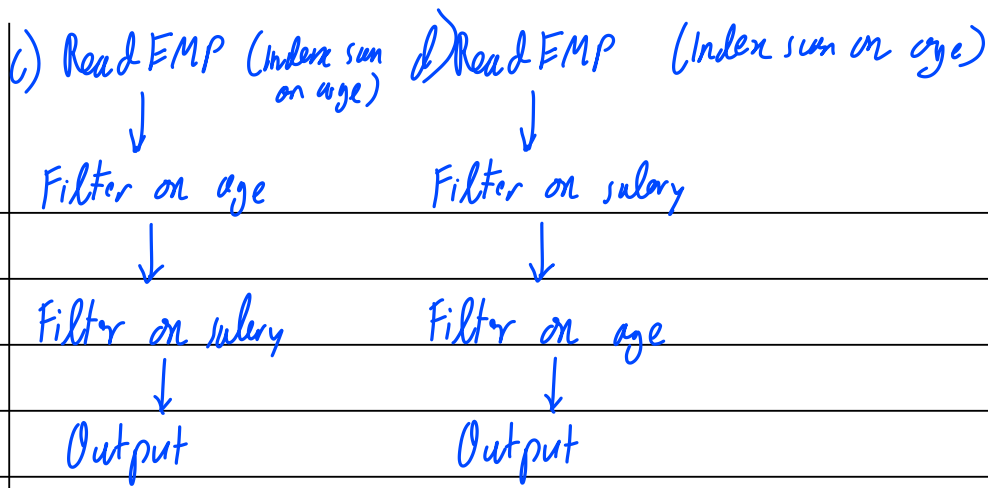
$$\begin{aligned} \text{retention factor on salary} &= \frac{100,000 - 40,000}{100,000 - 20,001} \approx \frac{60,000}{80,000} \approx \frac{3}{4} \\ \text{retention factor on age} &= \frac{1}{60 - 21 + 1} = \frac{1}{40} \end{aligned}$$

Hence no. of tuples in result = $100,000 \left(\frac{3}{4}\right) \left(\frac{1}{40}\right) = 1875$ tuples

2) The system R optimizer considers only two kinds of scans: sequential scan and index scan. The following trees shall be considered:

a) Read EMP (seq. scan) b) Read EMP (seq. scan)





The costs for each plan is as follows:

a) 1000 pgs (store in buffer)

b) 1000 pgs (store in buffer)

c) Because there is an unclustered index on age we will have to read those
 d) pages:

$$\begin{aligned}
 & (N_{\text{Pages}}(I) + N_{\text{Tuples}}(\text{EMP})) \times \text{P.F.S} \\
 &= (200 + 100,000) \left(\frac{1}{40}\right) \\
 &= 2505 \text{ pages}
 \end{aligned}$$

e) Because there is a clustered index on salary,

f) $\text{No. of pages} = (500 + 1000) \left(\frac{3}{4}\right) = 1125$

Plans e) & f) have the minimum disk accesses. Hence either plan e) or f) shall be selected.

2. The Dean of IISc asks you to estimate the cardinality of the query

select distinct (S.name, S.hostel) from STUDENT S

where the *STUDENT* relation has the schema [*SRNo*, *name*, *hostel*, *gpa*] and individual B^+ -tree indexes are available on the *SRNo* and *name* attributes, respectively.

Assume that you use, for this estimation, the strategy outlined in Figure 1 of the Sampling paper discussed in class. Within this framework, describe your: (a) partitioning approach, (b) random sampling strategy, and (c) computation technique for the *RandomSample()* function.

[Hint: Consider the possibility of result partitions with fractional sizes.] (6 marks)

- a) The partitioning approach is basically splitting the tuples of a table into sets called as partitions. Any two partitions should be disjoint, i.e. they must not contain any common elements. Furthermore, the union of all these partitions must yield the original set A . In mathematical terminology:

original set

$$A = P_1 \cup P_2 \cup \dots \cup P_n = \bigcup_{i=1}^n P_i \text{ and}$$

$$P_i \cap P_j = \{\} \quad \forall i \neq j$$

- b) We shall select n random pages from the table using the B^+ tree index on *SR.No*. The value of n shall be dynamic, as per the algorithm in Figure 1. We can do this because the query is a *SELECT* on a single table without any joins. Because of the *DISTINCT* keyword, we will not be able to take random tuples as samples. This is because a tuple will always have a distinct name & hostel. Hence, the smallest unit we can take as a sample will be a page.

- c) The *RandomSample()* returns the number of tuples that match the predicate. The *RandomSample()* function does this by checking the cardinality of the query, if the query were to be executed on that page only. It will then return the number of tuples matching the predicate.

This would have been more accurate if we had a buffer. We could have taken n pages and stored them in the buffer. We

would then run the query on the page, in the buffer for a more accurate estimate.

Since mention of a buffer (cache) is not present in the question, I have assumed the presence of a single page buffer.

Estimating the cardinality: Estimated cardinality = $\frac{sn}{m}$, where:

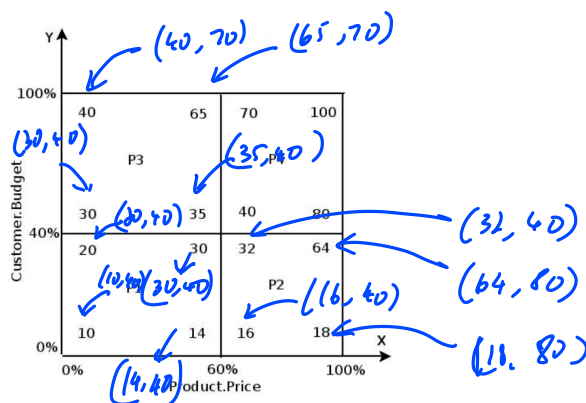
s = no. of tuples in sample that match the predicate

m = no. of tuples used for sampling.

n = total no. of tuples in the table.

3. In the Linear Hashing paper, it is claimed that the the address computation algorithm A2 is extremely efficient since the *while* loop is on average executed at most *twice*. Do you agree? If yes, prove the claim. If no, indicate *your* assessment of the average number of iterations. (For simplicity, assume that the file construction parameters, N and K , are both set to 1.) (4 marks)

4. Consider the plan diagram *PD* shown in the following figure, annotated with the corner costs for each of the optimality regions. For this diagram, is it possible to identify the minimum λ at which the reduced plan diagram (using Cost-Bounding approach) is guaranteed to feature only a *single* plan? If yes, identify (with justification) this minimum value, and also mention the associated plan. If not, explain why it is infeasible.



[Note: Assume plan cost monotonicity (PCM) is strictly adhered to by all plan cost functions.] (4 marks)

Because a plan can only be "swallowed" by a plan to the top right, only P4 shall be able to swallow every one. For this, we need to find a λ s.t. $(10\lambda + 1)(1 + \lambda) < \text{new-cost}$

$\therefore$ min value of λ should be s.t. $18(1 + \lambda) = 80$

$$\lambda = \frac{80 - 18}{18} = \frac{62}{18} \approx 3.5$$

$\therefore \lambda$ must be 350%.

5. You are given the task of designing a range-encoded bitmap index on an integer attribute *Weight* that records the weight of a package to be mailed. The minimum possible weight is 101 grams and the maximum allowable weight is 1100 grams. Disk space is very scarce and you have only enough to store 40 bitmaps overall.

1. What is the minimum number of components required in the index? Give a set of base values that satisfy the space constraints. (3 marks)
2. What is the minimum number of components required to produce a *time-optimal* index that satisfies the space constraints? Give a set of base values for this index and calculate the expected time consumption incurred by the index. (3 marks)

1) Cardinality of the field, $C = 1100 - 101 + 1 = 1000$

1-component : $\langle 40 \rangle \Rightarrow$ Not possible

2-component : $\langle 20, 20 \rangle$

(cardinality covered) $= 20^2 = 400$

∴ Not possible

3-component : $\langle 13, 13, 14 \rangle$

cardinality $= 13 \times 13 \times 14 = 2366$

* We know that the minimum cardinality that can be covered by n components and 40 bitmaps $= \binom{40}{n}^n$

Therefore, the minimum no. of components is 3. Available set of base values is $\langle 13, 13, 14 \rangle$.

2) For time-optimality, we must reduce the no. of components & each component must store as many bitmaps as possible. Without the 40-bitmaps space constraint the ideal bitmap would be $\langle 1000 \rangle$. However, because we have the space constraint, we must be using at least three components (as proven in (1)). For this, we need to make one component as large as possible & other components as small as possible:

Let $b_1 = 2$ then for optimality, max. possible cardinality = $\langle 2, 19, 19 \rangle$
(min value) $= 2 \times 19^2$

$= 722$ (won't work)

Let $b_1 = 3$, then for max possible cardinality = $\langle 3, 18, 19 \rangle$
 $= 3(18)(19)$

$= 1026 \checkmark$

Hence, with $b_1 = 3$, we can fulfil the cardinality requirement.

To further maximise time efficiency, we can further tune b_2 & b_3 :

b_2 can be replaced with $b_2 - \delta$, and b_3 can be replaced by $b_3 + \delta$,

$$\text{where } \delta = \max \left\{ 0, \left\lfloor \frac{b_2 - b_3 + \sqrt{(b_2 + b_3)^2 - 4C/b_1}}{2} \right\rfloor \right\}$$

$$= \max \left\{ 0, \left\lfloor \frac{(11 - 19) + \sqrt{(18 + 19)^2 - 4(\frac{10000}{3})}}{2} \right\rfloor \right\}$$

$$= \max \left\{ 0, \left\lfloor \frac{-1 + 5.47}{2} \right\rfloor \right\}$$

$$= \max \{ 0, \lfloor 2.47 \rfloor \}$$

$$= 2$$

$\therefore$ The most time optimal index (with 40 bitmaps) is $\langle 3, 16, 21 \rangle$

$$\text{Total bitmaps} = 3 + 16 + 21 = 40 \checkmark$$

$$\text{cardinality} = 2(16)(21) \cdot 1008 \geq C \checkmark$$

Assuming uniform (equal) probabilities across operators,

$$\text{Total expected time} = 2 \left[n - \sum_{i=1}^n \frac{1}{b_i} + \frac{1}{3} (b_1 - 1) \right]$$

$$= 2 \left[3 - \frac{1}{3} - \frac{1}{16} - \frac{1}{21} + \frac{1}{3} \left(\frac{1}{21} - 1 \right) \right]$$

$$= 2(2.24)$$

≈ 4.48 runs on average

6. Consider the ARIES recovery algorithm on the execution log shown below. The master record points to the checkpoint at LSN 10. Assume that the dirty page table written in the checkpoint record (at LSN 20) at the beginning is empty.

LSN	LOG
00	update: T1 writes P1
10	begin-checkpoint
20	update: T1 writes P2
30	end-checkpoint Transaction Table: T1,
	Dirty Page Table:
40	update: T2 writes P3

50	update: T3 writes P4
60	T1 commit
70	update T3 writes P5
80	T1 end
90	update T2 writes P3
100	T2 abort
	CRASH, RESTART

P₁ has been flushed as of begin-checkpoint

- At the end of the analysis phase, what are the contents of the transaction table and the dirty page table? Show the lastLSN field for the active transactions and the recLSN field for the dirty pages. (2 marks)
- Show the first two CLR's written during the UNDO phase. Show the undoNextLSNs. (2 marks)
- Due to some glitch in the power supply, the system keeps rebooting repeatedly and the recovery algorithm runs on each restart. Specify the maximum number of log records that may be cumulatively written during recovery over all the restarts. (2 marks)

T T		DPT	
Xid	lastLSN	Page ID	recLSN (prevLSN)
T₁	00 20 60	P ₁	20
T ₂	40 90	P ₃	40 90
T ₃	50 70	P ₄	50
		P ₅	70

2) Once the analysis phase is complete, we obtain the DPT & TT.

Then, we REDO all transactions, starting from the smallest $nextLSN$ (or $firstLSN$) in the DPT.

Once the REDO ALL is done, we UNDO all losers. Loser transactions are transactions that haven't committed yet. In this case, we shall have to undo T_1 & T_3 .

As we know, undo LOGERS happens in reverse order (i.e. larger LSN first).

To Undo List

90 $\Rightarrow$ Undo the update; Write CLR log; Add prev LSN (40)

70 $\Rightarrow$ Undo the update, " " " " (50)

40

50

This question explicitly asks for the first two CLR's. These CLR's are those written corresponding to LSN s 90 & 70.

90 $\Rightarrow$ Undo T_2 LSN 90; undo next LSN = 40

70 $\Rightarrow$ Undo T_3 LSN 70; undo next LSN = 50

3) Analysis Phase: No log records written

Redo ALL " " " " " "

Undo losers " : One CLR is written to LOG for every LSN that is "undone"

Because we have to undo 4 LSN s, we will have a maximum of 4 CLR's being written, even in the case of many successive failures after in a short period of time.